SkillX

Project Review, Technical Reference & Viva Guide

A peer-to-peer skill-swapping mobile application — Flutter + Firebase

Feature-complete • 13 phases • backend deployed, seeded & live • 10 Cloud Functions

Firebase project: skillx-9a717 • Region: us-central1

Contents

1. Project Overview

2. Technology Stack — Languages & Tools Explained

3. Folder Structure

4. How the App Runs (Startup & Request Flow)

5. Architecture & Design Patterns

6. Core Concepts Explained

7. Phase-by-Phase Technical Breakdown

8. Key Algorithms & Logic

9. Data Model (Cloud SQL Tables)

10. Project Review / Viva — Questions & Answers

11. Glossary of Terms

1. Project Overview

SkillX is a peer-to-peer learning application. A learner finds a peer who *teaches* a skill they *want to learn*, sends a request, meets over an automatically generated Google Meet link, then takes an AI-generated quiz to earn badges, build a daily streak, progress along a structured learning path, and finally receive a participation certificate.

It is entirely peer-to-peer: every user is both a learner and a teacher, and there are no paid instructors. The app is built with **Flutter** (one codebase for Android and iOS) and a **Firebase** backend.

The core loop

- **Discover** — browse skills and the peers who teach them.
- **Request & Session** — request a teacher; on accept a chat opens and a video session with a Google Meet link is scheduled.
- **Quiz** — after the session the teacher generates an AI quiz; the learner attempts it and is graded on the server.
- **Reward** — badges (bronze/silver/gold), a day-streak, and progress along a learning path.
- **Certificate** — completing every module of a path issues a participation certificate (PDF).

2. Technology Stack — Languages & Tools Explained

2.1 Programming languages

Dart	The whole mobile app (everything under app/lib)	The language Flutter uses. Compiled, strongly typed, good for UI.
TypeScript	Cloud Functions (functions/src)	JavaScript with types; runs on Node.js on the server; catches errors at compile time.
GraphQL (GQL)	Data Connect operations (dataconnect/*.gql)	A query language for the database API. You ask for exactly the fields you want; it is compiled to SQL.
SQL (PostgreSQL)	The actual database (Cloud SQL: skillx-pg / skillxdb)	The relational database where all structured data physically lives.

2.2 Frameworks & platform

Flutter	Google’s UI toolkit. You build the screen out of "widgets" (small building blocks like Text, Row, Column). One codebase runs on Android and iOS.
Firebase	Google’s backend-as-a-service. Provides auth, databases, storage, messaging and serverless functions so we do not run our own servers.
Node.js 22	The runtime that executes the (TypeScript-compiled) Cloud Functions on Firebase.

2.3 Firebase services used

Firebase Auth	Sign-up / sign-in (email+password and Google), email verification, password reset.
Data Connect	A layer that lets us write GraphQL and stores the data in Cloud SQL (PostgreSQL). Generates a type-safe Dart SDK.
Cloud SQL (Postgres)	The relational database (users, skills, sessions, quizzes, attempts, paths, certificates, ratings).
Cloud Firestore	A realtime NoSQL database — used ONLY for chat and unread counters (the "realtime layer").
Firebase Storage	File storage for profile photos, resume PDFs and certificate PDFs.
Cloud Messaging (FCM)	Push notifications to the phone even when the app is closed.
Cloud Functions	Serverless TypeScript functions for privileged / secret server logic (grading, sessions, notifications, certificates).

2.4 External APIs

Google Gemini (gemini-2.5-flash)	AI generation of 5 multiple-choice quiz questions from the covered subtopics (via the @google/genai SDK, with a structured JSON response schema).
Google Calendar API	Creating a calendar event with a Google Meet link and emailing invites to both people (via the googleapis SDK + OAuth2).

2.5 Key Flutter packages (app/pubspec.yaml)

flutter_riverpod	State management (providers the UI watches).
go_router	Declarative navigation + the auth-gated redirect.
firebase_core / _auth / cloud_firestore / firebase_storage / firebase_messaging / firebase_data_connect	Firebase client plugins.
google_sign_in	Google sign-in button.
fl_chart	The dashboard line & bar charts (pure Dart).
pdf / printing	Generate the certificate PDF on-device and share it.
file_selector	Pick a resume PDF (chosen over file_picker for build compatibility).
image_picker	Pick a profile photo.
http	Call the Cloud Functions over HTTPS (instead of the cloud_functions plugin).
url_launcher	Open the Meet link / certificate URL.
intl	Date/number formatting.
google_fonts	Typography.

2.6 Node packages (functions/package.json)

firebase-functions	Declare the HTTPS onRequest functions.
firebase-admin	Admin access to Auth, Firestore, Data Connect, Messaging (bypasses security rules).
@google/genai	Gemini API client for quiz generation.
googleapis	Google Calendar API client.
nodemailer	Send generic notification emails over Gmail SMTP.

3. Folder Structure

The repository has three top-level parts: the Flutter app, the Cloud Functions, and the Data Connect schema/operations.

```
SkillX/
  app/                      Flutter mobile app
    lib/
      main.dart             app entry point
      core/
        router/             app_router.dart (go_router + auth redirect)
        theme/              app_colors.dart, app_spacing.dart
        widgets/            shared building blocks (buttons, etc.)
      shared/widgets/       skill_chip, user_avatar, ...
      generated/dataconnect/  auto-generated type-safe DB SDK (do not edit)
      features/             ONE folder per feature (see layering below)
        auth/ home/ discovery/ requests/ profile/ skills/ quiz/
        rewards/ paths/ dashboard/ certificate/ notifications/
        chat/ ratings/ about/
    test/                   unit + widget tests
    android/                Android build config (AGP/Gradle/Kotlin pinned)
  functions/
    src/
      index.ts              the 10 Cloud Functions
      dataconnect.ts        admin GraphQL helpers (executeRead / executeWrite)
      gemini.ts             Gemini quiz generation
      calendar.ts           Google Calendar + Meet link
      notify.ts             notify() push helper (toggles + quiet hours)
      email.ts              nodemailer notifier
  dataconnect/
    schema/schema.gql       every @table = a Postgres table
    connector/
      queries.gql           client-callable reads
      mutations.gql         client-callable writes
    seed_skills.gql         one-off catalogue seed
  firebase.json             wiring for all services
  docs/                     this guide + architecture + dev plan
```

Feature layering (used in every feature/ folder)

Each feature is split into four layers so responsibilities stay separate and testable:

Domain	domain/	Plain Dart model classes (no Flutter, no Firebase). e.g. PathDetail, NotificationPrefs, ChatMessage.
Data	data/	Repositories that talk to the outside world (Data Connect, Firestore, Cloud Functions over HTTP).
Application	application/	Riverpod providers & controllers — the state the UI watches.
Presentation	presentation/	Screens and widgets (the actual UI).

4. How the App Runs (Startup & Request Flow)

4.1 Startup sequence

When the app launches, this happens in order:

```
1. main() runs  -&gt;  WidgetsFlutterBinding.ensureInitialized()
2. Firebase.initializeApp(...)             connect to project skillx-9a717
3. runApp( ProviderScope( child: SkillXApp ) )
       ProviderScope = the root that holds all Riverpod state
4. SkillXApp builds MaterialApp.router( routerProvider )
5. go_router shows /splash first
6. The redirect reads appAuthStateProvider (a state machine):
       unknown          -&gt; stay on /splash (still checking)
       unauthenticated  -&gt; /login
       needsVerification-&gt; /verify-email
       needsProfile     -&gt; /complete-profile (Google onboarding)
       ready            -&gt; /home
7. On /home the FCM device token is registered and notification
   taps are wired to deep-link into the app.
```

The redirect is re-run automatically whenever auth state changes, because a small ChangeNotifier bridge listens to `appAuthStateProvider` and tells go_router to re-evaluate.

4.2 Example end-to-end flow: sending a learn request

```
User taps "Send request" on a teacher's skill
  -&gt; SendRequestSheet calls requestsController.sendRequest(...)
     -&gt; FunctionsRepository.sendRequest(...)  (HTTP POST + Firebase ID token)
        -&gt; Cloud Function sendRequest (server):
             * verifyIdToken() -&gt; who is calling
             * request_insert (admin GraphQL -&gt; Cloud SQL)  [new row]
             * find the teacher's name + skill name
             * notify(teacherUid, "teaching", ...) capped 3/day
                 -&gt; FCM push to the teacher's phone
Teacher opens the app -&gt; Home shows the incoming request
  -&gt; taps Accept -&gt; acceptRequest Cloud Function:
       * status = ACCEPTED
       * opens a Firestore chat document (with member cards)
       * notify the learner ("request accepted")
```

This shows the two ways SkillX talks to the backend: **Data Connect** (GraphQL to SQL, for reads and simple writes) and **Cloud Functions** (HTTPS, for privileged writes that also send notifications / open chats).

5. Architecture & Design Patterns

5.1 The hybrid database (why two databases?)

Structured, relational data (users, skills, sessions, quizzes, attempts, paths, certificates, ratings) lives in **Cloud SQL (PostgreSQL)** through Data Connect — it is type-safe and schema-validated. Only **chat and unread counters** live in **Firestore**, whose realtime listeners are the right tool for live messaging. Using each database for what it is best at is the "hybrid" design.

5.2 Client → server security model

- Cloud Functions are HTTPS **onRequest** functions (not onCall — the onCall path pulled a plugin that broke the Android build).
- The app sends the user's **Firebase ID token** in the Authorization header; the function calls `verifyIdToken()` to know who is calling.
- Sensitive writes (grading, streak, path unlock, certificates, ratings average) are **server-only**. The correct-answer field of a quiz never leaves the server.
- Data Connect operations are scoped with `auth.uid` so a user can only read/write their own rows.

5.3 Patterns used

Repository	Each feature has a repository class that hides "where the data comes from" (Data Connect / Firestore / HTTP) behind simple methods.
Provider (Riverpod)	State is exposed as providers; screens watch them and rebuild automatically on change.
Layered / Clean-ish	domain -> data -> application -> presentation keeps UI, logic and I/O separate.
Code generation	The Data Connect SDK and (optionally) Freezed models are generated, not hand-written, so they always match the schema.
Single source of truth	Server-owned aggregates (averageRating, streak) are computed on the server, never trusted from the client.

6. Core Concepts Explained

6.1 State — Riverpod

State is the data the UI is currently showing and reacting to (is it loading? which questions are on screen? the current streak). **Riverpod** is the library that manages it: you declare *providers* (named data sources) and screens *watch* them, so only the widgets that depend on a piece of state rebuild when it changes.

```
final myLearningPathsProvider = FutureProvider(...);   // a source of state
final async = ref.watch(pathDetailProvider(pathId));   // a screen watches it
async.when(loading: ..., error: ..., data: (path) =&gt; ...);  // reacts to it
```

6.2 Push — Firebase Cloud Messaging (FCM)

A **push notification** is a message the server sends to a phone even when the app is closed (the banner on your lock screen). **FCM** is Google's free delivery service. On sign-in the app registers a device *token* (its address); when something happens server-side, a Cloud Function calls `notify()` which sends via FCM; tapping the notification deep-links into the right screen. (Chat, by contrast, uses Firestore streams for live updates while the app is open.)

6.3 Splash

The **splash screen** is the first screen at launch (usually just the logo) shown while the app does startup work. In SkillX it is shown for the moment it takes to decide: is someone logged in? is their email verified? have they finished onboarding? — then it redirects to Home, Login, Verify-email or Complete-profile.

6.4 Session rows

A **session** is one scheduled teaching meeting. A **row** is one record in a database table. When a teacher submits the pre-session form, the server writes **1 row in the Session table** (the meeting) plus **1 row per learner in the SessionLearner table** (each attendee). "Session + SessionLearner rows" simply means "it inserts those database records".

6.5 Mandatory empty state, from immutable attempt history (Phase 9)

- **Empty state** = what a screen shows when there is no data yet. A new user has zero quizzes, so the dashboard shows a friendly "No quiz activity yet" message instead of blank charts. The design brief makes this **mandatory** — the empty case must be designed, not left blank.
- **Attempt history** = every quiz taken, one QuizAttempt row each.
- **Immutable** = never changed after writing. Re-taking a quiz adds a NEW row; it never edits or deletes the old one. All dashboard charts are computed purely from this permanent history, so the numbers are trustworthy and grading cannot be retroactively faked.

6.6 SQL vs GraphQL (GQL)

What	The language of relational databases (Postgres)	An API query language that sits in front of a database
You ask in	Tables, rows, columns, JOINs	Types and fields (a graph)
Example	SELECT name FROM users WHERE uid='1'	query { user(id:"1") { name } }
Result	Flat rows	Nested JSON of exactly the fields you asked

In SkillX: data physically lives in **Cloud SQL (Postgres)**, but you never write SQL. **Firebase Data Connect** lets you write **GraphQL** (the .gql files) and compiles it into SQL under the hood. So the .gql files are GraphQL; skillx-pg / skillxdb are the SQL database those become.

7. Phase-by-Phase Technical Breakdown

Each phase was built, static-analysed (flutter analyze) and Android-build verified before moving on.

Phase 1 — Foundation

Project skeleton, theme tokens (`app_colors.dart`, `app_spacing.dart`), shared widgets, the go_router shell, the Data Connect schema, and the generated Dart SDK.

Phase 2 — Authentication

Feature `features/auth`: splash, register (with photo + skill pickers), login, forgot-password (with a resend countdown), email-verify gate, and Google sign-in + complete-profile onboarding.

Key idea: a state machine `AuthStatus` (unknown / unauthenticated / needsVerification / needsProfile / ready) drives the go_router redirect via `appAuthStateProvider`.

Phase 3 — Home & Discovery

Home header with streak, tappable search, upcoming-session card, incoming accept/reject, and browse-offerings with a category filter. Skill-detail screen with a send-request sheet.

New Data Connect reads: `ListTeachOfferings`, `SearchTeachOfferings`.

Phase 4 — Profile & Resume

Profile screen (identity, stat tiles, three skill sections, resume card, certificates strip), edit-profile, manage-skills (saves add/remove diffs). Resume PDF picked with `file_selector` and stored in Storage.

Phase 5 — Requests -> Session -> Meet

Cloud Functions begin. `acceptRequest` (ACCEPTED + opens chat + notifies) and `createSession`.

Updated later: `createSession` now creates a Google Calendar event with a Meet link and emails invites — the teacher no longer pastes a link. The app calls functions over HTTPS with a Firebase ID token.

Phase 6 — Quiz + AI Generation + Approval

`functions/gemini.ts`: `generateQuizQuestions` uses Gemini with a structured response schema to draft 5 MCQs from the covered subtopics + level.

Functions `generateQuiz`, `approveQuiz`, `gradeQuiz` (server-side scoring; the correct-answer index never leaves the server; one immutable QuizAttempt; badge gold≥80 / silver≥60 / bronze≥40).

Phase 7 — Rewards, Badges, Streak

The streak is computed server-side inside `gradeQuiz` using the device's local date: same day = unchanged, next day = +1, any gap = reset to 1. Rewards screen: current/longest streak cards + a badge grid (tiers + 7/30/100-day milestones).

Phase 8 — Learning Paths & Unlocking

Lock/unlock is driven entirely by `UserProgress.unlockedUpTo`: passing a module's quiz (≥40%) sets unlockedUpTo = N+1 and marks the path complete on the last module — written server-side in `gradeQuiz`. `enrollPath` creates the progress row.

Phase 9 — Dashboard

Four summary cards, an `fl_chart` line chart (last 10 scores), a per-skill bar chart (top 6), a recent-activity list, and a mandatory empty state — all derived from the immutable QuizAttempt history.

Phase 10 — Certificate

On path completion, `issueCertificate` writes an immutable Certificate record (server-computed average, idempotent); the app generates the PDF on-device (the pure-Dart `pdf` package, participation-only wording), uploads it to Storage, and links it via `attachCertificatePdf` (write-once). Celebration screen + share.

Phase 11 — Notifications

One server helper `notify(uid, category, {title,body,route})` skips muted categories and quiet hours and attaches a deep-link route. A settings screen (per-category toggles + quiet hours) saves to User.notificationPrefs. The interest-targeted "learners waiting to teach" push (capped 3/day per teacher) fires from `sendRequest`.

Phase 12 — Chat

Realtime 1:1 over Firestore. `acceptRequest` seeds the chat doc with member cards + an unread map; `createSession` drops in a session-link card. A conversation list (unread badges) and a thread (realtime bubbles, composer, unread-clears-on-open) with Start (teacher) / Join (learner) on the Meet link.

Phase 13 — Ratings, About, Tests

`submitRating` (Cloud Function) upserts a 1-5 star rating per learner+session and recomputes the teacher's average server-side; rated from a sheet in the chat thread. An About screen. Domain unit tests (notification prefs, path lock-state, dashboard aggregates) + widget/token smoke tests — flutter test green.

8. Key Algorithms & Logic

8.1 Auth-gate redirect (a state machine)

A single enum decides where every navigation lands; the redirect maps each state to a route and re-runs whenever auth changes.

```
switch (authStatus) {
  unknown           -&gt; hold on /splash
  unauthenticated   -&gt; /login
  needsVerification -&gt; /verify-email
  needsProfile      -&gt; /complete-profile
  ready             -&gt; /home
}
```

8.2 Streak computation (server, device-local date)

```
computeStreak(last, today, current):
  if last is null            -&gt; 1                (first ever)
  if last == today           -&gt; unchanged        (already counted today)
  yesterday = today - 1 day
  if last == yesterday       -&gt; current + 1      (consecutive day)
  else                       -&gt; 1                (streak broken, restart)
```

The client sends its local date (YYYY-MM-DD) so the "day" follows the user’s timezone. longestStreak = max(longestStreak, newStreak).

8.3 Quiz grading (server-side; answers never trusted)

```
grade(quiz, answers):
  score = count of answers[i] == correctIndex[i]
  percentage = score / total * 100
  badge = gold  if pct &gt;= 80
          silver if pct &gt;= 60
          bronze if pct &gt;= 40
          none   otherwise
  insert ONE immutable QuizAttempt row
  (correctIndex is read on the server only and never sent to the client)
```

8.4 Learning-path unlock

Progress is stored as a single number `unlockedUpTo`. For a module of order N:

```
state(module):
  completed  if pathComplete OR module.order &lt;  unlockedUpTo
  available  if module.order == unlockedUpTo
  locked     if module.order &gt;  unlockedUpTo

on passing module N (&gt;= 40%):   unlockedUpTo = min(N+1, totalModules)
if N == totalModules:           isPathComplete = true
```

8.5 Quiet hours (wrap-around time window)

```
inQuietHours(prefs):
  if start or end is null      -&gt; false (disabled)
  localHour = (nowUTC + utcOffsetMinutes) hour
  if start &lt; end   -&gt; start &lt;= hour &lt; end          (same day, e.g. 1-&gt;6)
  else             -&gt; hour &gt;= start OR hour &lt; end  (wraps midnight, 22-&gt;7)
```

The device’s UTC offset is stored with the preference so the server can evaluate the window in the user’s local time.

8.6 Dashboard aggregation (from immutable attempts)

- **Average score** = mean of all attempt percentages.
- **Best score** = max percentage.
- **Recent line chart** = the last 10 attempts, reversed to oldest→newest order.
- **Per-skill bar chart** = group attempts by skill, average each, sort high-to-low, keep the top 6.

8.7 Unread counters (Firestore)

```
sendMessage: batch {
  add message doc { senderId, text, sentAt }
  update chat { lastMessage, lastMessageAt, unread[other] += 1 }
}
openThread (markRead): chat.unread[me] = 0
```

8.8 "Waiting to teach" daily cap (3/day)

```
on sendRequest:
  insert the request
  today = UTC date
  counter = firestore notifCounters/{teacherUid}
  n = (counter.date == today) ? counter.teachCount : 0
  if n &lt; 3:
     if notify(teacher, "teaching", ...) actually sent:
        counter = { date: today, teachCount: n + 1 }
```

notify() returns whether a push was really sent (false if muted / quiet hours / no token), so the cap only counts real notifications.

8.9 Certificate issue (idempotent + server aggregate)

```
issueCertificate(pathId):
  require userProgress.isPathComplete
  if a certificate already exists -&gt; return it (immutable)
  averageScore = mean of the learner's attempts for the path's skill
  insert Certificate (immutable) ; set userProgress.certificateId
  (client then generates the PDF, uploads it, and links the URL once)
```

9. Data Model (Cloud SQL Tables)

User	uid (PK), name, email, photoUrl, currentStreak, longestStreak, lastActivityDate, totalSessions, averageRating, fcmToken, notificationPrefs, resume*
Skill	name (unique), category, description
UserSkill	(user, skill, relation) — HAVE / TEACH / LEARN, proficiency
Request	learner, teacher, skill, status (PENDING/ACCEPTED/REJECTED/COMPLETED)
Session / SessionLearner	the meeting (topic, level, meetingLink, optional module/path) and one row per attendee
Quiz / QuizQuestion	quiz metadata + 4 options; correctIndex is never sent to learners
QuizAttempt	user, quiz, score, percentage, badgeEarned — immutable, one per attempt
LearningPath / LearningModule	ordered modules per skill
UserProgress	(user, path) — unlockedUpTo, isPathComplete — server-write only
Certificate	user, path, averageScore, modulesCompleted, pdfUrl — server-write, immutable
Rating	session, learner, teacher, stars (1-5), feedback

Chat is NOT in SQL — it lives in Firestore as `chats/{id}` documents with a `messages` subcollection.

10. Project Review / Viva — Questions & Answers

The following are the kinds of questions a project reviewer / examiner would ask, grouped by area, with model answers.

A. General & Architecture

Q. What is SkillX and what problem does it solve?

A. SkillX is a peer-to-peer skill-swapping app: it connects people who want to learn a skill with peers who can teach it, and gamifies learning with quizzes, badges, streaks, learning paths and certificates. It removes the cost barrier of paid tutoring by making everyone both a learner and a teacher.

Q. Why did you choose Flutter?

A. One codebase compiles to both Android and iOS, it has a rich widget system for custom UI, hot-reload speeds up development, and it integrates cleanly with Firebase. It saved building and maintaining two native apps.

Q. Why Firebase instead of a custom backend?

A. Firebase provides auth, databases, storage, messaging and serverless functions out of the box, so there is no server to run or scale. It let a small team ship a full backend quickly and securely.

Q. Explain your overall architecture in one line.

A. A Flutter client using Riverpod + go_router, talking to a hybrid backend: Cloud SQL (via Data Connect / GraphQL) for structured data, Firestore for realtime chat, and TypeScript Cloud Functions for privileged logic, with Gemini and Google Calendar as external APIs.

Q. Why did you use two databases (Cloud SQL and Firestore)?

A. Each is used for what it is best at. Relational, structured data with relationships (users, sessions, quizzes) fits a SQL database with a strict schema. Chat needs instant realtime updates, which Firestore's listeners provide. Mixing them is a deliberate "hybrid" design, not an accident.

B. Flutter & Dart

Q. What is a widget?

A. In Flutter everything on screen is a widget — a small, composable building block (Text, Row, Column, Container). You build complex UI by nesting widgets. Widgets are immutable descriptions; Flutter rebuilds them efficiently when state changes.

Q. Difference between StatelessWidget and StatefulWidget?

A. A StatelessWidget has no internal changing data (it only depends on its inputs). A StatefulWidget keeps mutable state (e.g. a form's current input) and can call setState to rebuild. In SkillX most screens are ConsumerWidget/ConsumerStatefulWidget so they can also watch Riverpod providers.

Q. What is async/await and Future used for?

A. Network and database calls take time, so they return a Future (a value that will arrive later). await pauses until it resolves without blocking the UI. The UI shows loading / data / error states via Riverpod's AsyncValue.

C. State Management

Q. What is Riverpod and why use it over setState?

A. Riverpod is a state-management library. Instead of each widget owning and passing data manually, you declare providers (shared, testable sources of state) and widgets watch them; only dependent widgets rebuild. It decouples business logic from the UI and makes it testable.

Q. What is the difference between a Provider, FutureProvider and StreamProvider?

A. Provider exposes a plain value (e.g. a repository). FutureProvider exposes a one-off async result (e.g. load the dashboard). StreamProvider exposes a continuous stream of updates (e.g. the realtime chat messages).

Q. How does the UI know when to refresh after an action?

A. Providers are invalidated (e.g. ref.invalidate(myQuizzesProvider)) after a mutation; watchers re-run and rebuild with fresh data. Streams update automatically.

D. Backend, Functions & Security

Q. Why are some operations Cloud Functions and others direct database calls?

A. Simple, user-scoped reads/writes go straight to Data Connect (GraphQL). Anything privileged or secret — grading (must hide the answer key), computing streaks/averages, sending notifications, creating calendar events — runs in a Cloud Function so the client can never tamper with it.

Q. How do you secure the Cloud Functions?

A. They are HTTPS functions that require the caller's Firebase ID token in the Authorization header and verify it with verifyIdToken(). They then know the real uid and check ownership (e.g. only the request's teacher can reject it).

Q. How do you stop a learner from seeing the quiz answers?

A. The learner-facing query deliberately omits the correctIndex field, and grading happens on the server. The correct answers are only read inside the gradeQuiz function; they are never sent to the device.

Q. What are Firebase security rules?

A. Declarative rules on Firestore/Storage that decide who can read/write what. e.g. only a chat's participants can read its messages; a user can only write their own resume file. Cloud Functions use the admin SDK, which bypasses these rules for trusted server logic.

E. Database, SQL & GraphQL

Q. What is Data Connect?

A. A Firebase product that lets you define a schema and write GraphQL operations, which it compiles to SQL and runs against a managed Cloud SQL (PostgreSQL) instance. It also generates a type-safe Dart SDK so the app calls the database with compile-time safety.

Q. Difference between SQL and GraphQL?

A. SQL is the language the relational database itself speaks (tables, rows, JOINs). GraphQL is an API query language where you request exactly the fields you need and get nested JSON back. In SkillX we write GraphQL; Data Connect turns it into SQL against Postgres.

Q. What does "immutable QuizAttempt" mean and why does it matter?

A. Each attempt is written once and never edited or deleted; re-taking adds a new row. It gives a trustworthy, append-only history that the dashboard and certificates are computed from, and prevents scores being retroactively altered.

Q. What was the "insecure operations" warning during deploy?

A. Data Connect flags operations not scoped to auth.uid or that are public. Most of ours are intentional (public skill catalogue, discovery browsing), so we added an insecureReason note documenting the intent; the one real gap (reject request) was scoped to the teacher via auth.uid.

F. AI & External APIs

Q. How is the AI quiz generated?

A. The gradeQuiz teacher flow calls the generateQuiz function, which sends the topic + covered subtopics + difficulty to Google Gemini (gemini-2.5-flash) with a structured JSON response schema, so it returns exactly 5 questions each with 4 options and a correct index. The teacher can edit and approve before release.

Q. Why use a structured response schema for Gemini?

A. It forces the model to return valid, predictable JSON matching our shape (questionText, options[4], correctIndex), so we do not have to parse free text and risk malformed output.

Q. How are meetings created?

A. The createSession function uses the Google Calendar API (via OAuth2) to create an event with a Google Meet link and invite both people; Calendar emails the invites. The generated Meet link is stored on the session and shown in chat.

G. Features & Algorithms

Q. How does the day-streak work?

A. On completing a quiz the server compares the last-active date with today (the device's local date): same day = unchanged, consecutive day = +1, a gap = reset to 1. It is server-side so it cannot be faked.

Q. How does a learning path unlock modules?

A. Progress is a single number, unlockedUpTo. Modules below it are completed, the one at it is available, above it are locked. Passing a module's quiz with >= 40% advances unlockedUpTo; failing or skipping keeps it locked.

Q. How do you prevent notification spam to popular teachers?

A. The "a learner wants to learn your skill" push is capped at 3 per teacher per day using a Firestore counter document; the cap only counts pushes that were actually delivered (respecting the teacher's mute/quiet-hours settings).

Q. How is the teacher rating calculated?

A. A learner submits 1-5 stars via the submitRating function, which upserts one rating per learner+session and then recomputes the teacher's average from all their ratings server-side — the average is a server-owned value the client cannot set directly.

H. Testing, Build & Deployment

Q. What did you test?

A. Domain logic with unit tests (notification-prefs round-trip, learning-path lock-state, dashboard aggregation) plus widget/token smoke tests. flutter test passes. Business rules live in pure Dart classes precisely so they are testable without Firebase.

Q. Describe a hard bug you solved.

A. The Android build needed JDK 25, which forced a very new Gradle/AGP with "built-in Kotlin"; two popular plugins (file_picker, cloud_functions) would not compile, so we replaced them with file_selector and plain HTTPS calls. Also, deploying functions failed with a build service-account permission error because the organization disabled default IAM grants, fixed by granting the Cloud Build role.

Q. How is the app deployed?

A. firebase deploy pushes Data Connect (schema + connectors to Cloud SQL), Firestore/Storage rules, and the 10 Cloud Functions (a predeploy hook compiles TypeScript first). The skill catalogue is seeded once. Secrets (Gemini key, OAuth tokens) live in Secret Manager, never in the app.

Q. What would you add next?

A. A deeper accessibility pass, Firestore rules unit tests using the emulator, live-session presence, and richer teacher analytics. The architecture (feature layering + repositories) makes these additive.

11. Glossary of Terms

Widget	A Flutter UI building block; screens are trees of widgets.
Provider	A Riverpod source of state that widgets watch.
Repository	A class that hides where data comes from behind methods.
Future / async	A value that arrives later; await waits for it.
Stream	A sequence of values over time (e.g. live chat messages).
FCM	Firebase Cloud Messaging — push notifications.
Token (ID token)	A signed proof of who the logged-in user is, sent to the server.
Row	One record in a database table.
Immutable	Never changed after being written.
GraphQL	An API query language; you ask for exact fields.
SQL / Postgres	The relational database language / engine data lives in.
Data Connect	Firebase layer: write GraphQL, stored in Cloud SQL.
Firestore	Firebase realtime NoSQL database (chat only here).
Cloud Function	Serverless server-side code (TypeScript) on Firebase.
onRequest / HTTPS	A function you call over normal HTTP(S).
Deep-link	A notification that opens a specific screen in the app.
Idempotent	Safe to run more than once with the same result.
Aggregate	A value computed from many rows (average, count).
Seed	Load initial data (the 24 starter skills) into the database.

SkillX — Project Review, Technical Reference & Viva Guide. Generated for academic project review. Source of truth: docs/PROJECT_GUIDE.md.